

CS 577 Deep Learning

Homework 2: Inside an MLP

Irma Modzgyrshvili (Hawk username: A20597307)

Fall 2026

Part 0: Conventions and Reproducible Setup

0.1 Environment and public tests

Environment Item	Version / Value
Python	3.13.5
Platform	macOS-15.7.4-arm64-arm-64bit-Mach-O
NumPy	2.5.2
PyTorch	2.13.0
Required device	CPU

0.2 Shape and reduction contract

The targets are stored with shape $(B, 1)$ so they match the logits shape $(B, 1)$. If we combine an array with shape $(B, 1)$ and another with shape $(B,)$, NumPy broadcasting can produce an incorrect result with shape (B, B) . Because the loss is the mean over the batch,

$$J = \frac{1}{B} \sum_{i=1}^B \ell_i,$$

the factor $\frac{1}{B}$ enters backward propagation in

$$\frac{\partial J}{\partial S} = \frac{\sigma(S) - y}{B}.$$

Part 1: Forward and Backward Mathematics

1.1 Forward shape trace

Quantity	Expression / role	Shape
X	input batch	(B, D)
W_1, b_1	first affine parameters	$(D, H), (H,)$
Z_1	$XW_1 + b_1$	(B, H)
A_1	$\text{ReLU}(Z_1)$	(B, H)
W_2, b_2	output affine parameters	$(H, 1), (1,)$
S	logits	$(B, 1)$
y	binary targets	$(B, 1)$
J	mean batch loss	scalar

The first bias b_1 has shape $(H,)$ and is broadcast across all B examples when it is added to XW_1 , which has shape (B, H) .

The second bias b_2 has shape $(1,)$ and is broadcast across all B examples when it is added to A_1W_2 , which has shape $(B, 1)$.

The values in S are logits, or raw model outputs. They are not probabilities. The probability of class 1 is obtained using $\sigma(S)$.

Since J is the mean loss over the batch, it is a scalar.

1.2 BCE-with-logits derivative

For one example, let

$$p = \sigma(s).$$

The binary cross-entropy loss is

$$\ell(s, y) = -y \log p - (1 - y) \log(1 - p).$$

First, differentiate with respect to p :

$$\frac{\partial \ell}{\partial p} = -\frac{y}{p} + \frac{1 - y}{1 - p}.$$

Since

$$p = \sigma(s) = \frac{1}{1 + e^{-s}},$$

the sigmoid derivative is

$$\frac{\partial p}{\partial s} = p(1 - p).$$

Using the chain rule,

$$\frac{\partial \ell}{\partial s} = \frac{\partial \ell}{\partial p} \frac{\partial p}{\partial s}.$$

Therefore,

$$\frac{\partial \ell}{\partial s} = \left(-\frac{y}{p} + \frac{1-y}{1-p} \right) p(1-p).$$

Simplifying,

$$= -y(1-p) + (1-y)p$$

$$= -y + yp + p - yp$$

$$= p - y.$$

Since $p = \sigma(s)$,

$$\frac{\partial \ell}{\partial s} = \sigma(s) - y.$$

The full loss is the mean over the batch,

$$J = \frac{1}{B} \sum_{i=1}^B \ell_i,$$

so the first backward signal is

$$\frac{\partial J}{\partial S} = \frac{\sigma(S) - y}{B}.$$

1.3 Complete backward chain

Starting from

$$dS \equiv \frac{\partial J}{\partial S},$$

we backpropagate through the output affine layer,

$$S = A_1 W_2 + b_2.$$

The output-layer weight gradient is

$$dW_2 = A_1^T dS,$$

with shapes

$$A_1^T : (H, B), \quad dS : (B, 1),$$

so

$$dW_2 : (H, 1).$$

The output bias gradient is

$$db_2 = \sum_{i=1}^B dS_i,$$

so

$$db_2 : (1,).$$

The gradient passed back to the hidden activations is

$$dA_1 = dSW_2^T,$$

with

$$dS : (B, 1), \quad W_2^T : (1, H),$$

therefore

$$dA_1 : (B, H).$$

Next, since

$$A_1 = \text{ReLU}(Z_1),$$

the ReLU derivative gives

$$dZ_1 = dA_1 \odot \mathbf{1}(Z_1 > 0),$$

so

$$dZ_1 : (B, H).$$

For the first affine layer,

$$Z_1 = XW_1 + b_1.$$

The first-layer weight gradient is

$$dW_1 = X^T dZ_1,$$

with

$$X^T : (D, B), \quad dZ_1 : (B, H),$$

so

$$dW_1 : (D, H).$$

The first-layer bias gradient is

$$db_1 = \sum_{i=1}^B dZ_{1,i},$$

therefore

$$db_1 : (H,).$$

Bias gradients reduce over the batch axis because the same bias parameter is shared by all examples in the batch. The contributions from all examples are summed, while the remaining dimensions match the original bias parameter shapes.

Part 2: NumPy Forward and Backward Implementation

2.1 Stable primitives

The sigmoid was implemented using

$$\sigma(z) = \exp(-\text{logaddexp}(0, -z)),$$

which avoids directly computing large values of e^{-z} for very negative inputs.

The ReLU function was implemented elementwise as

$$\text{ReLU}(z) = \max(0, z).$$

Binary cross-entropy was computed directly from logits using the numerically stable expression

$$\ell(s, y) = \max(s, 0) - ys + \log(1 + e^{-|s|}).$$

The loss function first verifies that the logits and targets have identical shape $(B, 1)$, and then returns the mean loss as a Python scalar.

For an extreme-logit check, I used

$$z = \begin{bmatrix} -1000 \\ 1000 \end{bmatrix}, \quad y = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

The stable sigmoid returned

$$\sigma(z) = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

and the BCE-with-logits loss was

$$J = 0.0.$$

The computation remained finite with no overflow, infinity, or NaN values.

2.2 Fixed-case forward pass

For the supplied fixed check case, the cached shapes were

$$X \in \mathbb{R}^{4 \times 2}, \quad Z_1 \in \mathbb{R}^{4 \times 3}, \quad A_1 \in \mathbb{R}^{4 \times 3}.$$

The output logits had shape

$$S \in \mathbb{R}^{4 \times 1}.$$

The computed logits were

$$S = \begin{bmatrix} 0.358 \\ 0.840 \\ -0.131 \\ 0.850 \end{bmatrix}.$$

The minimum absolute hidden pre-activation value was

$$\min |Z_1| = 0.2.$$

2.3 Fixed-case manual backward pass

Parameter	Gradient shape	Gradient norm
W_1	(2,3)	0.3310189075461203
b_1	(3,)	0.1432627161908632
W_2	(3,1)	0.28557793465207953
b_2	(1,)	0.11372132946422508

The fixed-case mean BCE-with-logits loss was

$$J = 0.803402148481108.$$

All gradient norms are finite and nonzero, which indicates that the gradient propagates through both affine layers and the ReLU activation. In this check case, W_1 has the largest gradient norm, while b_2 has the smallest.

Part 3: Two Independent Gradient Checks

3.1 Centered finite differences

Parameter	Maximum absolute error	Relative error
W_1	1.205e-11	2.219e-11
b_1	6.612e-12	2.525e-11
W_2	4.716e-12	8.690e-12
b_2	1.862e-13	8.185e-13

3.2 PyTorch autograd comparison

Parameter	Maximum absolute error	Relative error
W_1	2.7755575615628914e-17	5.692702109414681e-17
b_1	5.551115123125783e-17	2.2195591008792773e-16
W_2	5.551115123125783e-17	1.202675990198298e-16
b_2	8.326672684688674e-17	3.6609986551693917e-16

The very small errors in both gradient checks show that my manual gradients are almost the same as the gradients from finite differences and PyTorch autograd. This means that the backward-pass calculations for W_1 , b_1 , W_2 , and b_2 are most likely correct for this fixed test case.

However, this does not prove that the whole program is correct in every case. It only checks the gradient calculations for this example. It does not check other parts, such as training, prediction, or different input data.

Part 4: Nonlinear Experiment

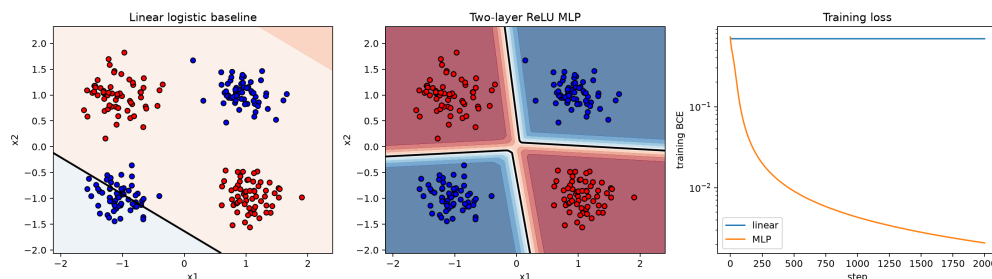
4.1 Linear baseline

The linear logistic regression model achieved a training loss of 0.6896 and training accuracy of 69.44%. On the test set, the loss was 0.7163 and the accuracy was 58.33%.

The linear model uses one affine score followed by a sigmoid function, so it creates only one straight decision boundary. The XOR data has alternating class regions, so one straight line cannot separate them well.

4.2 Manual MLP

The MLP used 8 hidden ReLU units, full-batch gradient descent, 2000 steps, learning rate 0.2, and seed 577. The training loss was 0.00208 with 100% accuracy. The test loss was 0.00184 with 100% accuracy.



4.3 Interpretation

The hidden ReLU units create new nonlinear features from the original inputs. The output layer is linear in these hidden features because it uses a weighted sum of them. The MLP learned the XOR pattern well, while the linear model underfit because it could only create one straight boundary. The similar training and test results do not show clear overfitting in this run. However, one run does not guarantee the same result for different initializations or hyperparameters.

Part 5: Reflection

Most useful cached value. The most useful cached values were Z_1 and A_1 . A_1 was needed to compute the gradient for W_2 , and Z_1 was needed to apply the ReLU derivative during the backward pass.

First gradient-check investigations. If the gradient check failed, I would first check the tensor shapes, the factor $1/B$ in the mean loss, and the ReLU derivative. I would also check if the original parameters were restored correctly after each finite-difference perturbation.

Why close is not bit-identical. The manual gradients, finite-difference gradients, and PyTorch gradients can be extremely close but not exactly identical because floating-point calculations have small rounding errors. Finite differences also use an approximation based on a small value of ϵ .

Assistance and Generative-AI Disclosure

TODO